# Raspberry Bin

Software Test Document (STD)

Security vulnerability, mitigation and test report

Team 27 – Computer Systems Project 2026–2027

Mentors: Dan Ploeşteanu, Timo van den Berg

Version 1.0 – October 28, 2026

Repository: https://gitlab.utwente.nl/computer-systems-project/2026-2027/Student_Groups/Pi5/Project_Group_27/Main

# Contents

**1 Introduction** **2**
1.1 Purpose . . . 2
1.2 Scope . . . 2
1.3 Related requirements (from the RAD) . . . 2

**2 System overview** **2**

**3 Identified vulnerability** **3**
3.1 V1: Brute-force guessing of the administrator password . . . 3

**4 Mitigation plan** **4**

**5 Test plan (Table 1)** **5**
5.1 How to run the tests . . . 5

**6 Test log (Table 2)** **7**

**7 Team review and AI tool use (Table 3)** **8**

**8 Conclusion** **8**

**A Test evidence** **9**

# 1 Introduction

## 1.1 Purpose

This document describes one concrete security vulnerability of the Raspberry Bin, the mitigation we implemented against it, and the tests that show that the mitigation works. It complements the Requirements Analysis Document (RAD) and the System Design Document (SDD).

## 1.2 Scope

The tests cover the web-facing part of the system:

- the **administrator login** of the web dashboard (`POST/api/admin/login`) and the admin-only routes behind it (`/api/admin/*`);
- the **scan endpoint** used by the bin itself (`POST/api/scan`), which accepts a camera image and returns the waste category.

The mechanical parts (servos, lids, solenoid locks) and the Raspberry Pi software are out of scope for this document.

## 1.3 Related requirements (from the RAD)

- **[H] Security Requirement – Administrator access:** users must log in as an administrator before manually operating lids or changing settings; the system checks the administrator's permission on *every* protected request, while regular users may scan without logging in.
- **[M] Security Requirement – Wrong password requests:** after 5 wrong attempts the system blocks further login attempts for 5 minutes, but scanning and sorting must keep working during the block.
- **[H] Functional Requirement 6 – Rejection of waste:** if an item cannot be classified, all lids stay closed.

# 2 System overview

The Raspberry Bin consists of four parts (see the SDD for the full design):

1. **Bin (Raspberry Pi 4B + camera):** takes a photo of the item in the scanning area and sends it to the backend with a device API key.
2. **Backend API (Go, Gin):** receives scans, validates the request, stores the image in Firebase Storage, asks the ML service for a category, saves the result in Firestore and tells the bin which lid to open (`can_sort`). It also serves the admin login. It runs behind a Caddy reverse proxy that provides HTTPS.
3. **ML service (Python, FastAPI, ConvNeXt-Small):** classifies the image into PMD, paper, organic, general waste or unsupported. It is only reachable from the backend on an internal Docker network, not from the internet.
4. **Web dashboard (Vue.js, in development):** lets an administrator log in and manually open or close compartments. The admin login and session check tested in this document are already implemented; the lid-control routes are being added behind the same `RequireAdmin` protection.

**Assets to protect.** (1) manual control of the lids, which must only be available to administrators; (2) the scan history and stored images; (3) the availability of scanning and sorting for normal users.

**Actors.** Regular user (scans an item, no login), administrator (logs in to the dashboard), the bin device (authenticates with an API key), and a potential attacker on the internet.

# 3 Identified vulnerability

## 3.1 V1: Brute-force guessing of the administrator password

**Description.** The admin dashboard is reachable from the internet through `POST/api/admin/login`. Without protection, an attacker can send an unlimited number of login requests and try passwords from a wordlist (brute force or credential stuffing with passwords leaked from other sites) until one works. Because each request is a small JSON message, thousands of guesses per minute are possible.

**Why it matters for the Raspberry Bin.** A successful guess gives the attacker a valid admin session. With it they could open any lid at any time (defeating the sorting, letting people throw anything into a compartment, or interfering with someone's hands near a moving lid), change settings and read the scan history. This directly breaks the RAD requirement that only administrators may operate lids.

**Attack steps.**

1. Find the login endpoint (it is used by the public web dashboard).
2. Send many `POST/api/admin/login` requests with a known username (e.g. `admin`) and different passwords.
3. Optionally measure response times to find out which usernames exist.
4. Use the returned session token to call the protected admin routes.

**Risk assessment.**

| | |
|---|---|
| Likelihood | **High** without mitigation: the endpoint is public and automated tools for password guessing are freely available. |
| Impact | **High**: full manual control of the bin and access to its data. |
| Overall | **High** – must be mitigated before deployment. |

# 4 Mitigation plan

All measures below are **implemented** in the backend (branch `main`) and are covered by the tests in Section 5.

| # | Measure | How it works | Where |
|---|---|---|---|
| M1 | Account lock after 5 failed attempts | Each wrong password increases a counter stored with the admin account. On the 5th wrong password the account is locked for 5 minutes; during the lock even the *correct* password is refused (HTTP 429). A successful login resets the counter. | `internal/auth/service.go` (`MaxFailedAttempts`, `LockDuration`) |
| M2 | Rate limit on the login endpoint | Token bucket per client IP: 1 request every 3 seconds with bursts of up to 5; extra requests get HTTP 429. Only Caddy is trusted for `X-Forwarded-For`, so an attacker cannot fake their IP. | `internal/middleware/ratelimit.go`, `cmd/api/main.go` |
| M3 | Strong, slow password hashing | Passwords are stored only as bcrypt hashes (cost 12, random salt), which makes offline guessing slow. New admin passwords must be 12–72 characters with letters and digits. Admins can only be created on the server with the `create-admin` tool, not through the API. | `internal/auth/password.go`, `cmd/create-admin` |
| M4 | No username enumeration | For an unknown username a dummy bcrypt check is still done, so a wrong username takes as long as a wrong password and returns the same error message. | `internal/auth/service.go` (`dummyHash`) |
| M5 | Secure sessions, checked on every request | After login the client gets a random 32-byte token; only its SHA-256 hash is stored. Sessions expire after 8 hours and can be ended with logout. The `RequireAdmin` middleware validates the token on every admin request. | `internal/auth/token.go`, `internal/middleware/admin.go` |
| M6 | Audit logging | Every failed, locked and successful login is logged with username and client IP, so an attack can be noticed. | `internal/handlers/admin.go` |
| M7 | Scanning unaffected by the lock | Login and scanning are separate routes with separate limits and authentication (device API key), so a locked admin account never blocks sorting (RAD requirement). | `cmd/api/main.go` |

Implemented mitigation measures for V1.

**Why this is enough.** With M1 and M2 an attacker can try at most 5 passwords per account every 5 minutes, i.e. about 1 440 guesses per day instead of millions. Combined with the 12-character password policy (M3) this makes online guessing practically impossible, and M4 prevents the attacker from even learning which usernames exist.

**Residual risk and future work.** An attacker who knows the username can deliberately lock the admin out for 5 minutes at a time (denial of service on the dashboard; scanning keeps working because of M7). Possible improvements: a second factor for admins (the RFID card already mentioned in the RAD), or locking per (username, IP) pair instead of per username. The rate limiter is kept in memory, which is fine for our single backend instance but would need a shared store if the backend is ever scaled to several instances.

A second residual risk is a *race condition* in M1: the failed-attempt counter is read, increased and written back without a database transaction. If many wrong passwords arrive at exactly the same moment, some of them can be counted only once, so an attacker could try a few more than 5 passwords before the lock starts. M2 limits this to 5 parallel requests per IP address, but an

attacker with many IP addresses could still use it. The planned fix is to update the counter inside a Firestore transaction, so that every attempt is counted.

# 5 Test plan (Table 1)

| ID | Type | What is tested | Covers | Expected result |
|---|---|---|---|---|
| UT-1 | Unit test (Go) | `TestLoginLocksAfterFiveFailedAttempts`: four wrong passwords, a fifth wrong password, then the correct password just before and just after the 5-minute lock. | V1 / M1 | Attempts 1–4: `ErrInvalidCredentials`; attempt 5: `ErrAccountLocked`; correct password at 4 min 59 s: still `ErrAccountLocked`; correct password after 5 min: login succeeds. |
| MT-1 | Manual (API) | Brute-force attempt against the running backend: wrong passwords sent to `/api/admin/login`, then the correct password during the lock, while the bin keeps scanning. | V1 / M1, M2, M6, M7 | Attempts 1–4: HTTP 401; attempt 5: HTTP 429 "too many failed attempts"; correct password during lock: HTTP 429; scan during lock: HTTP 200; correct password after 5 min: HTTP 200 with a token; failed attempts appear in the log. |
| MT-2 | Manual (API) | *Different risk:* malicious or invalid input on the scan endpoint `/api/scan`: request without API key, a text file renamed to `.jpg`, a file larger than 5 MB, and a path-traversal `bin_id`. | Device auth and input validation | No key: HTTP 401; fake image: HTTP 400 "image must be JPEG or PNG"; 6 MB file: HTTP 413/400 "image is too large"; `bin_id=../../etc`: HTTP 400; a real JPEG: HTTP 200 with a category. |

Table 1 – Tests linked to the vulnerability and mitigations.

Supporting automated tests that already exist in the repository: `TestRequireAdmin`, `TestSessionExpires`, `TestLogoutEndsSession`, `TestCreateAdminStoresHashNotPassword`, `TestRateLimiterBlocksAfterBurst`, `TestAPIKeyAuth`, `TestDetectImageTypeRejectsFakeImage`, `TestValidateBinID`, and in the ML service `test_corrupted_file_is_rejected` and `test_low_confidence_cannot_be_sorted`. Statement coverage of the security-related backend packages (`go test ./... -cover`): `auth` 88.0 %, `clients` 90.0 %, `middleware` 86.2 %. The scan handler (`handlers`, 21.8 %) talks to Firebase and the ML service, so it is covered by the manual test MT-2 instead of unit tests.

## 5.1 How to run the tests

**UT-1.** The test uses an in-memory store and a fake clock, so it needs no Firestore and does not have to wait 5 real minutes:

```
cd backend
go test ./internal/auth -run TestLoginLocksAfterFiveFailedAttempts -v
go test ./...          # all backend tests
```

**MT-1.** Start the stack (`docker compose up` in `backend/`), create a test admin with `create-admin`, then:

```
URL=https://<domain>           # or http://localhost:8080 locally
for i in 1 2 3 4 5; do
  curl -s -o /dev/null -w "attempt $i: %{http_code}\n" -X POST $URL/api/admin/login \
       -H "Content-Type: application/json" \
       -d '{"username":"testadmin","password":"wrong-password-1"}'
  sleep 3                       # stay under the IP rate limit, so we test the account lock
done
# correct password while locked -> expected 429
curl -i -X POST $URL/api/admin/login -H "Content-Type: application/json" \
     -d '{"username":"testadmin","password":"<correct password>"}'
# scanning still works during the lock -> expected 200
curl -i -X POST $URL/api/scan -H "X-API-Key: $DEVICE_API_KEY" \
     -F bin_id=bin-1 -F image=@bottle.jpg
# wait 5 minutes, then the correct password -> expected 200 + token
```

**MT-2.**

```
curl -i -X POST $URL/api/scan -F bin_id=bin-1 -F image=@bottle.jpg  # no key -> 401
echo "not an image" > fake.jpg
curl -i -X POST $URL/api/scan -H "X-API-Key: $DEVICE_API_KEY" \
     -F bin_id=bin-1 -F image=@fake.jpg  # -> 400
head -c 6000000 /dev/urandom > big.jpg
curl -i -X POST $URL/api/scan -H "X-API-Key: $DEVICE_API_KEY" \
     -F bin_id=bin-1 -F image=@big.jpg  # -> 413/400
curl -i -X POST $URL/api/scan -H "X-API-Key: $DEVICE_API_KEY" \
     -F "bin_id=../../etc" -F image=@bottle.jpg  # -> 400
curl -i -X POST $URL/api/scan -H "X-API-Key: $DEVICE_API_KEY" \
     -F bin_id=bin-1 -F image=@bottle.jpg  # -> 200
```

# 6 Test log (Table 2)

All tests were run on 28 October 2026. UT-1 ran locally (macOS, Go 1.27.1); MT-1 and MT-2 ran against the deployed server `https://raspberry-bin-27.swedencentral.cloudapp.azure.com` (Azure VM, Docker stack with Caddy, the Go API and the ML service). The terminal output is in Appendix A.

| ID | Date / tester / environment | Steps | Expected | Actual | Pass? | Mitigation verified / fix |
|---|---|---|---|---|---|---|
| UT-1 | 28-10-2026, Bohdan, local `go test` | Run the UT-1 command from Section 5.1 | Test passes: lock on 5th attempt, correct password refused during lock, accepted after 5 min | `-- PASS: TestLogin LocksAfter FiveFailed Attempts` | Passed | M1 verified |
| MT-1 | 28-10-2026, 12:19–12:25 and 12:35–12:38 CET (repeat), Bohdan, server | 1. 5 wrong logins, 3 s apart. 2. Correct password. 3. Scan with device key. 4. Wait 5 min, correct password. 5. Check API log. | 401, 401, 401, 401, 429; 429; scan 200; login 200 + token; `[AUDIT]` lines in the log | 401, 401, 401, 401, 429; correct password during the lock: 429; scan during lock: 200; correct password: 200 + token (end of lock after 5 min verified by UT-1, see Appendix A); all failed/locked attempts logged with username and client IP. | Passed | M1, M2, M6, M7 verified |
| MT-2 | 28-10-2026, 12:18 CET, Bohdan, server | 1. Scan without key. 2. Fake `.jpg`. 3. 6 MB file. 4. `bin_id=../../etc`. 5. Real JPEG. | 401; 400; 413/400; 400; 200 with category | 401; 400 “image must be JPEG or PNG”; 400 “image is too large”; 400 “invalid bin_id”; 200, category `UNSUPPORTED` (glass bottle), `can_sort=false` | Passed | API key auth, file-type and size checks, `bin_id` validation verified |

Table 2 – Test log.

**Fixes.** No fixes were needed: all tests passed on the first run.
**Notes.** In MT-2 step 3 the response was 400 and not 413: the 6 MB file still fits inside the 6 MB request body limit (5 MB image plus 1 MB for the other form fields), so it is rejected one step later by the 5 MB image size check. Both outcomes were expected. In MT-2 step 5 the test photo shows a glass bottle; glass does not belong to any of our four compartments, so `UNSUPPORTED` with all lids closed is the correct result (FR6).

# 7 Team review and AI tool use (Table 3)

| Member | Reviewed | Date | AI tools used | If yes, how |
|---|---|---|---|---|
| Sepehr | ✓ | 28-10-2026 | Yes | Advice on materials and design choices for the physical build and electronics; language editing of documents. |
| Mack | ✓ | 28-10-2026 | Yes | Advice on materials and design choices for the physical build and electronics; language editing of documents. |
| Omar | ✓ | 28-10-2026 | Yes | Advice on materials and design choices for the physical build and electronics; language editing of documents. |
| Artem | ✓ | 28-10-2026 | Yes | Advice on the classification model and dataset, occasional review of code I wrote; language editing of documents. |
| Ruslan | ✓ | 28-10-2026 | Yes | Advice on the classifier, occasional review of code I wrote; language editing of documents. |
| Bohdan | ✓ | 28-10-2026 | Yes | Mainly advice on the backend, occasional review of code I wrote; language editing of documents. |

Table 3 – Team review and AI tool use.

**Use of AI tools.** All team members used AI tools as an assistant. Artem, Ruslan and Bohdan used AI to ask for advice and explanations about their own part of the system (Bohdan mostly the backend, Artem and Ruslan the classification model) and to occasionally get a review of code they wrote. Sepehr, Mack and Omar used AI for advice on materials and design choices for the physical build and electronics. All of us sometimes used AI to improve the language of our documents. All requirements, design decisions, the physical build and the choice of the vulnerability and tests were made by the team; all code was written by the team members themselves, who can explain it; AI only reviewed it and gave advice.

# 8 Conclusion

The most relevant vulnerability for the Raspberry Bin's web interface is brute-force guessing of the administrator password, which would give an attacker manual control of the lids. We mitigated it with an account lock after 5 failed attempts, an IP-based rate limit, bcrypt password hashing with a strong password policy, protection against username enumeration and securely stored, expiring sessions that are checked on every request. The unit test UT-1 and the manual tests MT-1 and MT-2, run against the deployed server, confirm that these measures work and that scanning keeps working during an attack; no fixes were needed. As future work we propose a second factor for administrators (RFID) to remove the remaining lock-out risk, and a Firestore transaction for the failed-attempt counter to close the race condition.

# A Test evidence

### UT-1 (28-10-2026, local)

```
$ go test ./internal/auth -run TestLoginLocksAfterFiveFailedAttempts -v -count=1
=== RUN   TestLoginLocksAfterFiveFailedAttempts
--- PASS: TestLoginLocksAfterFiveFailedAttempts (0.01s)
PASS
ok      raspberry-bin-api/internal/auth 0.285s

$ go test ./... -cover -count=1
ok      raspberry-bin-api/internal/auth         coverage: 88.0% of statements
ok      raspberry-bin-api/internal/clients      coverage: 90.0% of statements
ok      raspberry-bin-api/internal/handlers     coverage: 21.8% of statements
ok      raspberry-bin-api/internal/middleware   coverage: 86.2% of statements
```

### MT-1 (28-10-2026, deployed server)

Wrong passwords, 3 s apart, then a scan during the lock (12:19 CET):

```
attempt 1 (wrong password): {"error":"invalid username or password"} [HTTP 401]
attempt 2 (wrong password): {"error":"invalid username or password"} [HTTP 401]
attempt 3 (wrong password): {"error":"invalid username or password"} [HTTP 401]
attempt 4 (wrong password): {"error":"invalid username or password"} [HTTP 401]
attempt 5 (wrong password): {"error":"too many failed attempts, try again later"} [HTTP 429]
scan during lock: {"bin_id":"bin-1","category":"UNSUPPORTED","confidence":0.94,
                   "can_sort":false,"reason":"unsupported_item"} [HTTP 200]
```

Login attempt with the intended admin password at 12:24:37 CET, 4 min 56 s after the lock started (run by Bohdan). As explained below, the stored password turned out to be different, so this run only shows that a locked account refuses every login; the repeat shows it with the real correct password:

```
{"error":"too many failed attempts, try again later"} [HTTP 429]
```

**Repeat of MT-1 for steps 2 and 4.** After the first run, the admin account had to be re-created with `create-admin`, because it had accidentally been created with a non-Latin keyboard layout, so the intended password did not match. To test the end of the lock on the real account, MT-1 was repeated (12:36 CET):

```
attempt 1 (wrong password): {"error":"invalid username or password"} [HTTP 401]
attempt 2 (wrong password): {"error":"invalid username or password"} [HTTP 401]
attempt 3 (wrong password): {"error":"invalid username or password"} [HTTP 401]
attempt 4 (wrong password): {"error":"invalid username or password"} [HTTP 401]
attempt 5 (wrong password): {"error":"too many failed attempts, try again later"} [HTTP 429]
locked at 12:36:28 CET (server log)
```

Correct password during the lock, at 12:37:44 CET (run by Bohdan):

```
{"error":"too many failed attempts, try again later"} [HTTP 429]
```

Correct password on the re-created account, before the repeat (12:35:57 CET, run by Bohdan):

```
{"expires_at":"2026-10-28T19:35:57.130452819Z","token":"<token>"} [HTTP 200]
```

That the lock ends automatically after exactly 5 minutes is verified by UT-1, which moves a simulated clock to 4 min 59 s (still locked) and to 5 min 01 s (login succeeds).

API log of the first run (server times are UTC, i.e. CET −1 h):

```
2026/10/28 11:19:26 [AUDIT] failed admin login for "admin" from 145.126.27.204
2026/10/28 11:19:30 [AUDIT] failed admin login for "admin" from 145.126.27.204
2026/10/28 11:19:33 [AUDIT] failed admin login for "admin" from 145.126.27.204
2026/10/28 11:19:37 [AUDIT] failed admin login for "admin" from 145.126.27.204
2026/10/28 11:19:41 [AUDIT] locked admin login attempt for "admin" from 145.126.27.204
2026/10/28 11:24:37 [AUDIT] locked admin login attempt for "admin" from 145.126.27.204
```

### MT-2 (28-10-2026 12:18 CET, deployed server)

```
1. Scan without API key          -> {"error":"unauthorized"} [HTTP 401]
2. Text file renamed to fake.jpg -> {"error":"image must be JPEG or PNG"} [HTTP 400]
3. 6 MB file big.jpg             -> {"error":"image is too large"} [HTTP 400]
4. bin_id=../../etc              -> {"error":"invalid bin_id"} [HTTP 400]
5. Real JPEG (glass bottle)      -> {"bin_id":"bin-1","category":"UNSUPPORTED",
                                     "confidence":0.94,"can_sort":false,
                                     "reason":"unsupported_item"} [HTTP 200]
```